Lecture #6: Atomic structures encoding

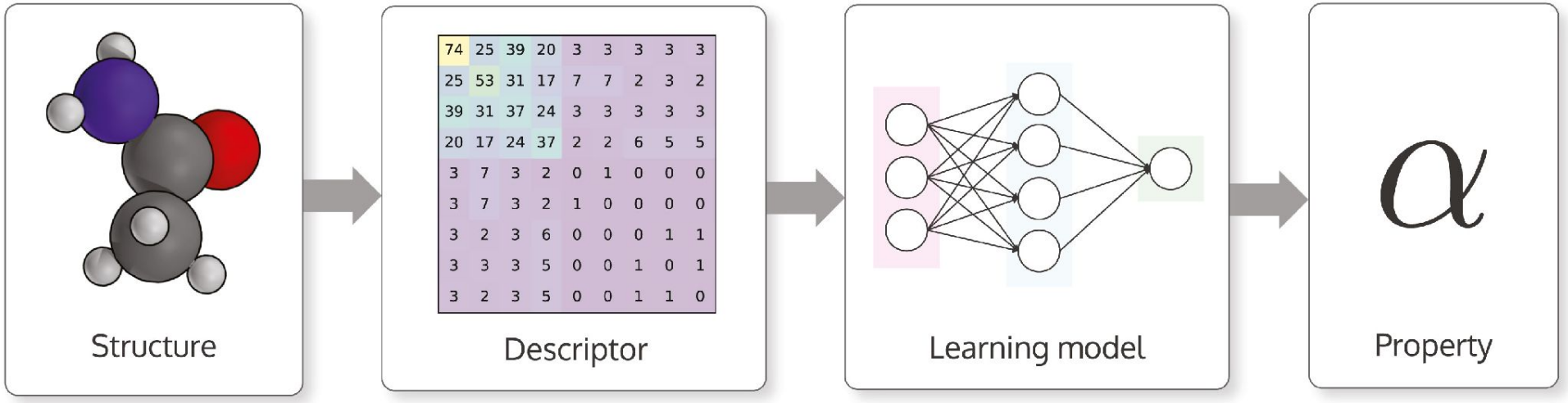
Slides by Artem Dembitskiy

Previously on

- Supervised machine learning
- Linear and ridge regression
- Random forest

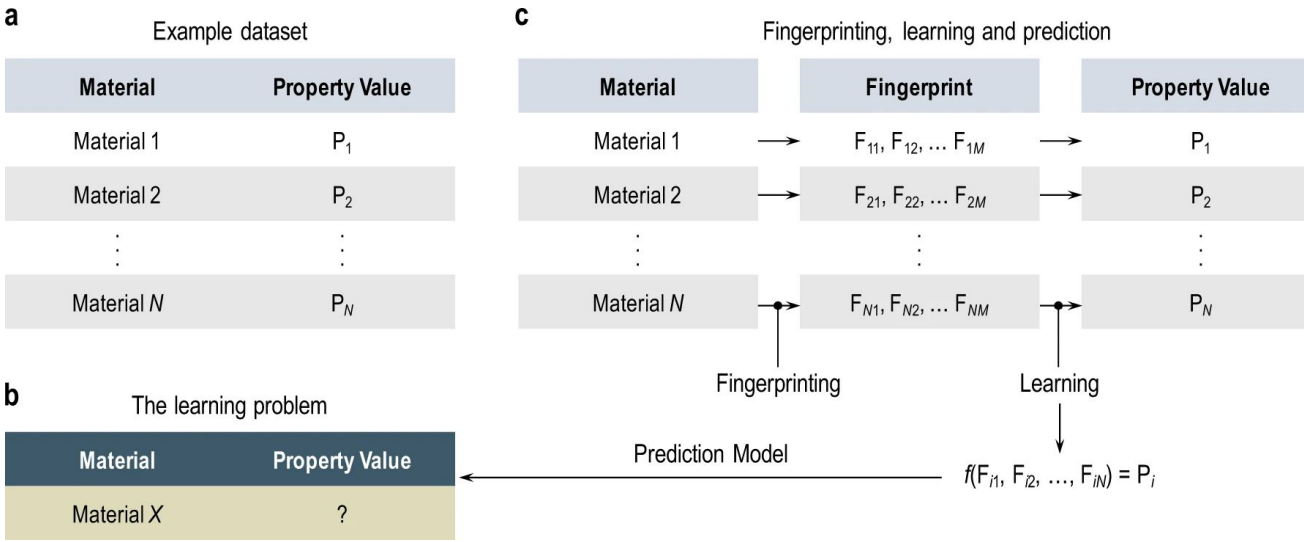
Goals/Agenda

- Hierarchy of crystal/molecular structure descriptors
- Feature importance
- Discussion of typical mistakes


74 25 39 20 3 3 3 3 3
25 53 31 17 7 7 2 3 2
39 31 37 24 3 3 3 3 3
20 17 24 37 2 2 6 5 5
3 7 3 2 0 1 0 0 0
3 7 3 2 1 0 0 0 0
3 2 3 6 0 0 0 1 1
3 3 3 5 0 0 1 0 1
3 2 3 5 0 0 1 1 0
Structure
Descriptor
Learning model
α
Property

Features

- In classical ML we use features
- to represent materials in a machine-readable format
- Think of it like a fingerprint

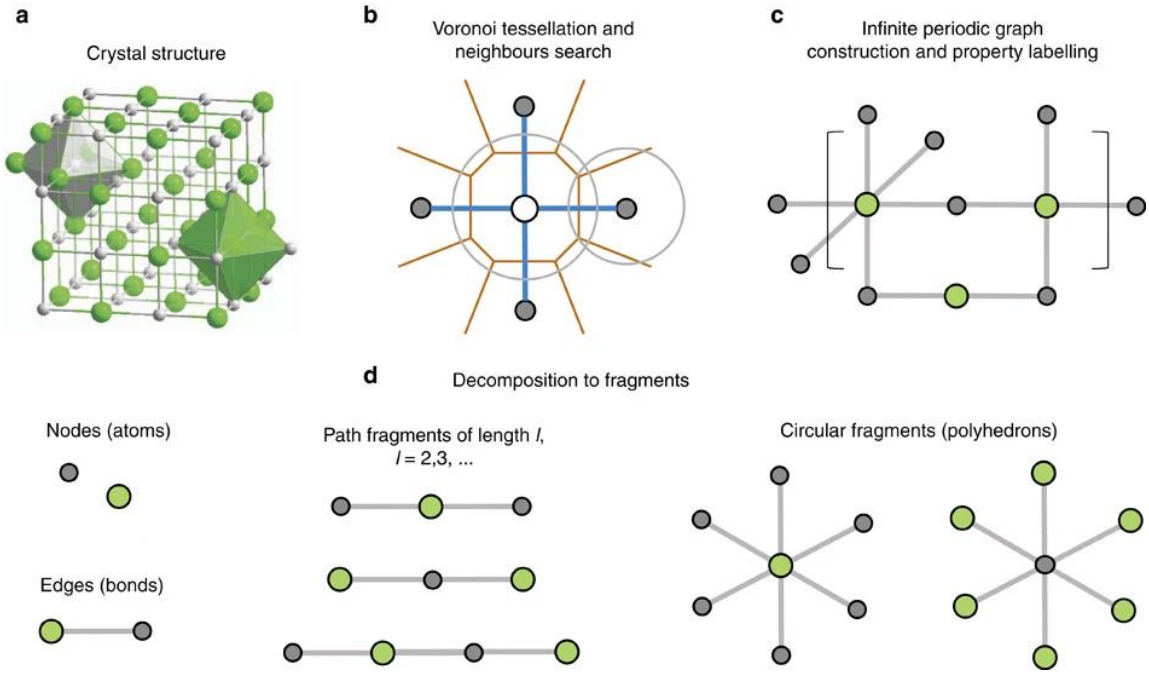


Effective descriptor design is one of the most important factors determining a model's predictive power.

Geometrical and compositional encoding of atomic structure

Hierarchy of features

- Depending on the resolution we have:
- Local descriptors
  - site
- Fragment descriptors
  - bond
  - polyhedron
- Global descriptors
  - chemical family
  - structural type
  - density



Geometrical (Structural) features

- Atomic packing (e.g. volume per atom, density)
- Voronoi polyhedra features
  - area, volume, face distance, solid angle
- bond distance/angle

Can be calculated for sublattices

Min/max/mean statistics

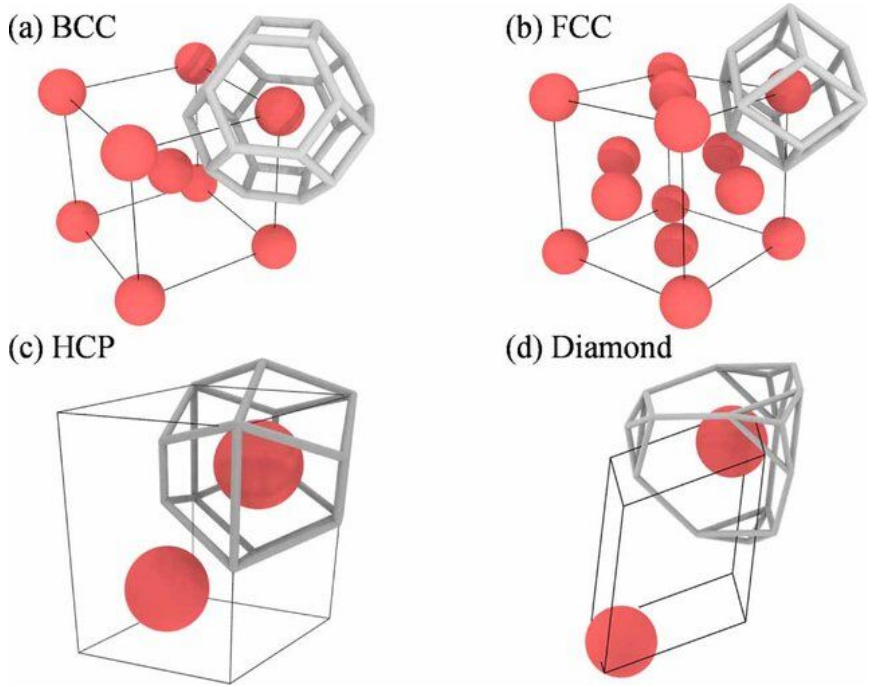


Image source

Voronoi polyhedra features can be collected using pymatgen

```
from pymatgen.analysis.local_env import VoronoiNN
features = VoronoiNN().get_voronoi_polyhedra(structure, site_id)
```

Image source

Compositional

(elemental/atomic) features

Aggregation over

- Atomic numbers
- Valence electrons
- Covalent radii
- Atomic fraction of each element in a composition
- Stoichiometry related
- Electron affinity
- Ionic radii
- Oxidation states
- Electronegativity
- …

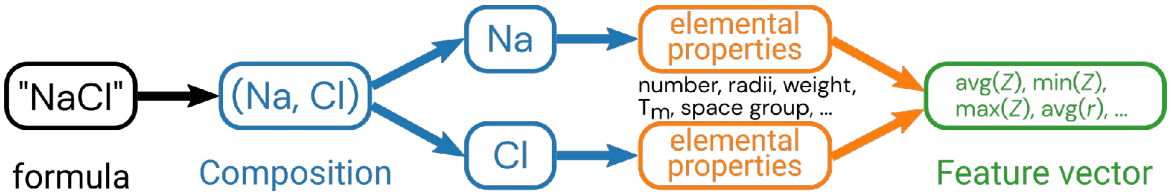


Image source

Atomic structure fingerprints

(not taught in the course)

- Atom-centered symmetry functions
- Smooth overlap of atomic positions
- Many-body tensor representations
- Pair distribution function
- X-ray diffraction pattern
- (U)MLIP embeddings

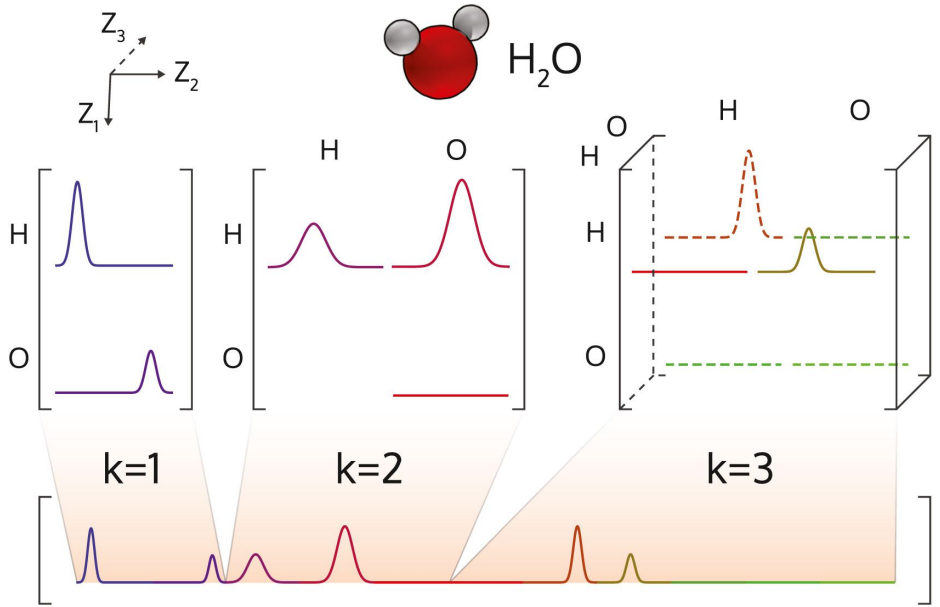
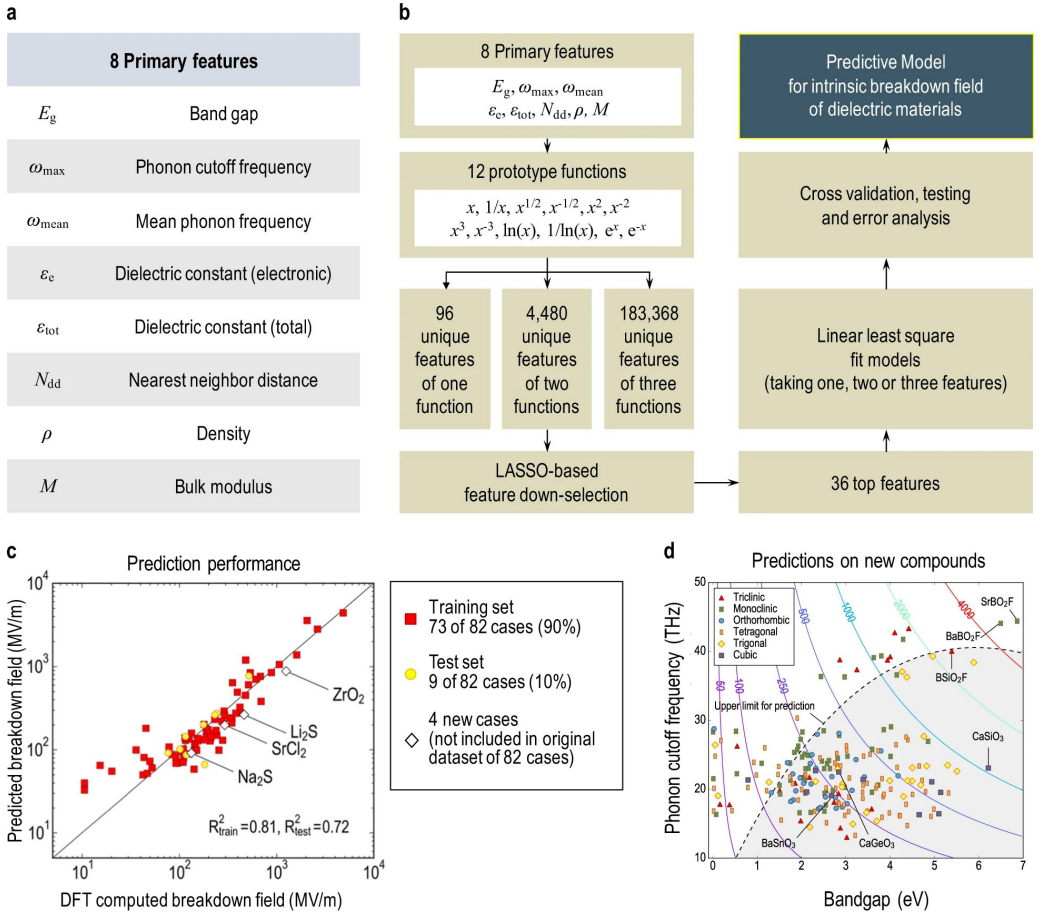


Image source

Feature engineering

- Primary descriptors are used to design more complex features
- e.g. by applying set of mathematical operators

See SISSO paper



Pros

- Better performance compared to primary descriptors

Cons

- Increased computational complexity
- Garbage feature needs to be filtered
- Lack of interpretability

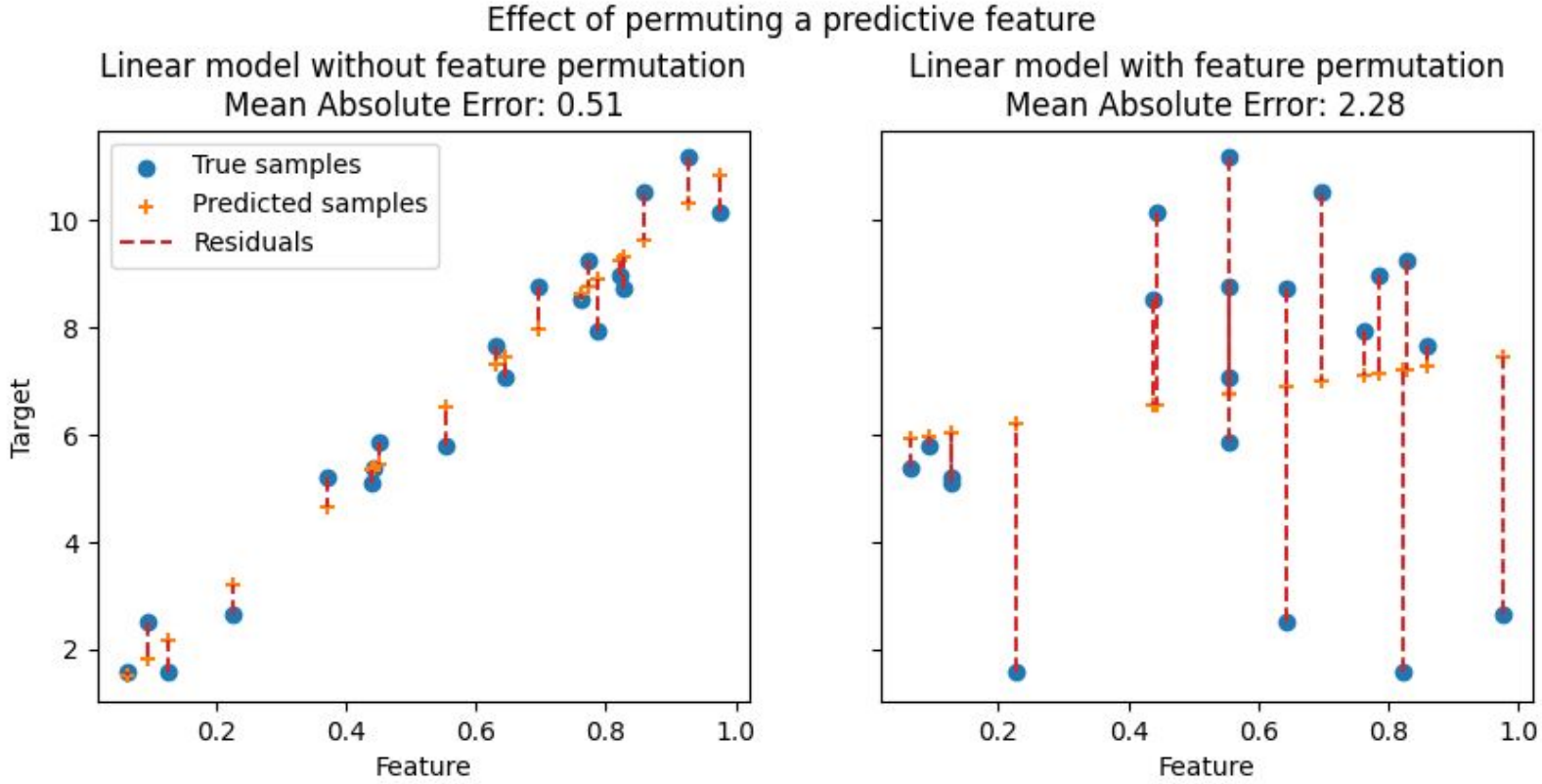
Permutation feature importance

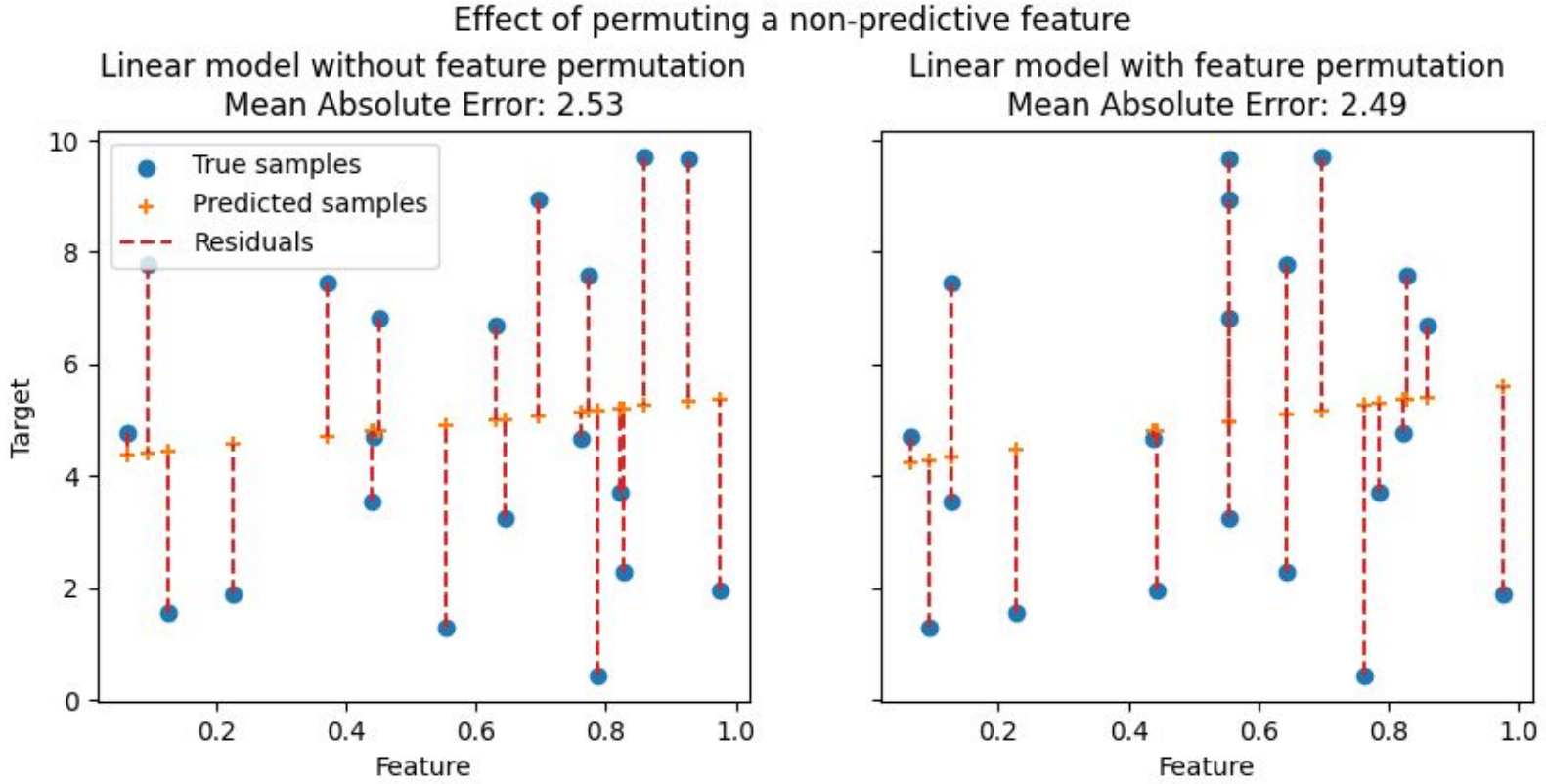
We want to know which features most influence model prediction

Given dataset {$\boldsymbol{X}$, y}

- Fit the model
- Get scores
- Randomly shuffle one of the feature vectors $\boldsymbol{x}_i$
- Refit model
- Get scores
- The higher degradation of the model performance the more important the feature

Permutation importance in sklearn


Effect of permuting a predictive feature
Linear model without feature permutation
Mean Absolute Error: 0.51
Linear model with feature permutation
Mean Absolute Error: 2.28
True samples
Predicted samples
Residuals
Target
Feature
Feature
2
4
6
8
10
0.2
0.4
0.6
0.8
1.0


Effect of permuting a non-predictive feature
Linear model without feature permutation
Mean Absolute Error: 2.53
Linear model with feature permutation
Mean Absolute Error: 2.49
True samples
Predicted samples
Residuals
Target
Feature
Feature
0
2
4
6
8
10
0.2
0.4
0.6
0.8
1.0

Pros

- Works for any model
- Yields variance of the feature importance

Cons

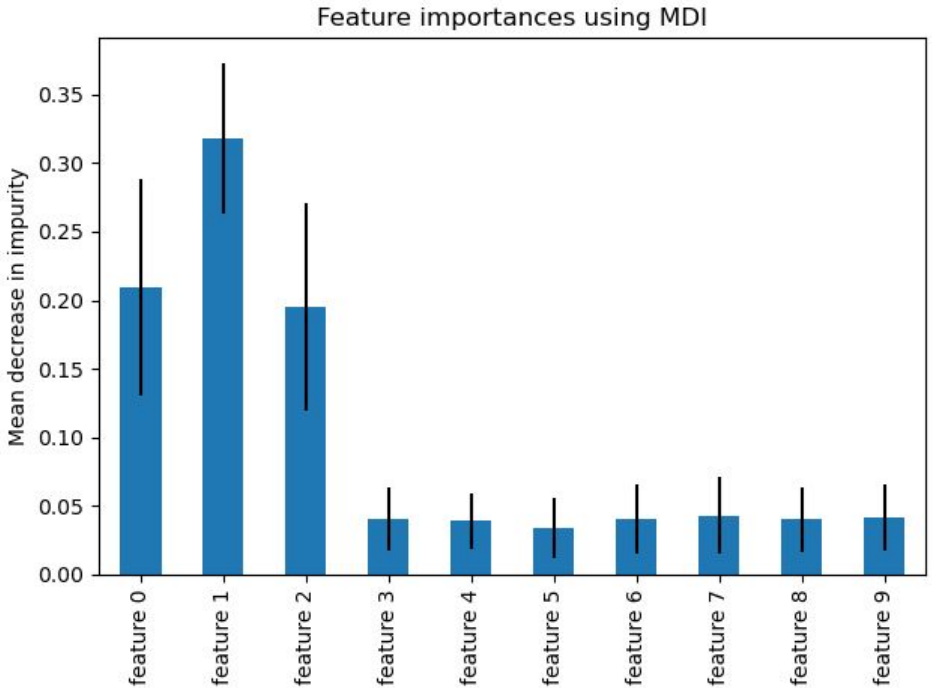
- Computational complexity
- Requires refitting the model for each feature
- Wrong output for correlated features

Intuition behind the impurity-based feature importance

Decision tree:

- Selects the best feature for splitting at each node
- Best split is measured by gain in purity (Gini impurity or Entropy)
- The higher gain the higher importance of the feature

In the case of Random Forest the feature importances are "computed as the mean and standard deviation of accumulation of the impurity decrease within each tree."



Feature importances with a forest of trees in sklearn

Pros

- You have this information after fitting the model
- Yields variance of the feature importance

Cons

- Wrong output for high-cardinality features

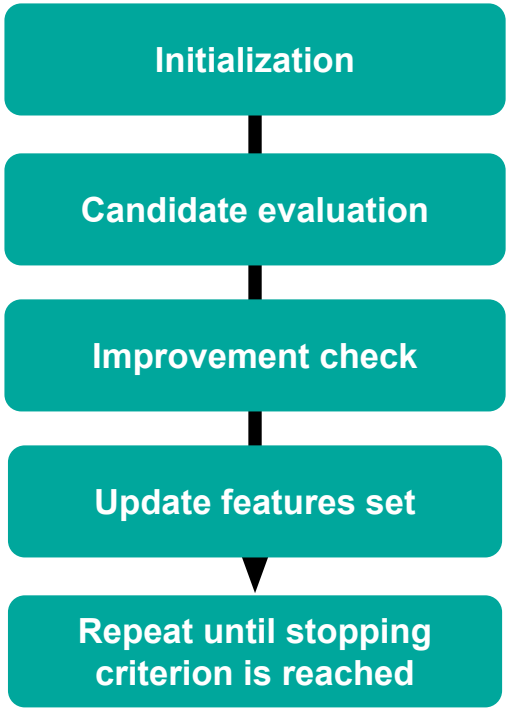
Feature selection

- Drop features with low variance
- Backward/Forward feature selection
- LASSO
- Tree-based feature selection

Why?

- Remove redundant features
- Develop robust model
- The less features the less compute is needed
- Feature collection = time = money
    - save your resources

Forward feature selection algorithm




```
Initialize: S = ∅, C = all features, best_score = −∞, stall = 0

While C ≠ ∅ AND stall < patience:
    best_f = None, current_best = −∞
    For each f in C:
        score = performance(S ∪ {f})
        If score > current_best:
            current_best = score, best_f = f

    If current_best > best_score + tolerance:
        S = S ∪ {best_f}, C = C \ {best_f}
        best_score = current_best, stall = 0
    Else:
        stall += 1
```

How to collect features

Design your own featurizer

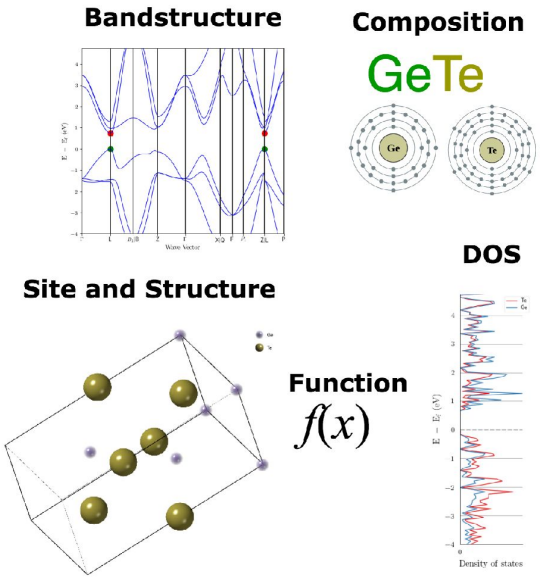
- You are an expert in your field
- Consider essential descriptors of your target
- Use pymatgen/ase/etc.

Ready-to-use Featurizers

- Matminer Python Library
- DScribe Python library

Gross level features

- Generate gross level features non-specific to your task
- Select the best candidates



pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$

Structure		Composition		Function	
Feature 1	Feature 2	Feature 3	Feature 4	Feature 1, 1/x	Feature 1, logx
0.1	1e-14	.003	223	10	-1
4.2	1.2e-12	.002	14	.238	.62
1.1	1e-6	.0031	101	.91	.041

The most important properties of an ideal descriptor*:

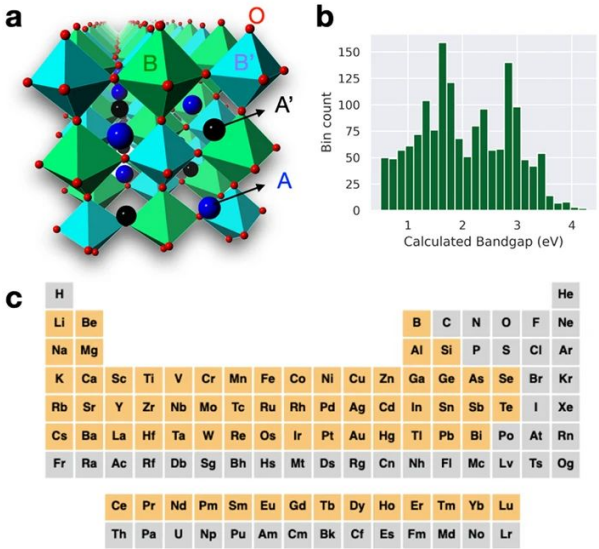
- Invariant with respect translation of the coordinate system
- Invariant with respect to rotation of the coordinate system
- Invariant with respect to permutation of atomic indices: changing the enumeration of atoms does not affect the target
- Unique: single way to construct a descriptor and the descriptor itself corresponds to a single property
- Continuous: small changes in the atomic structure -> small changes in the descriptor
- Compact
- Computationally cheap

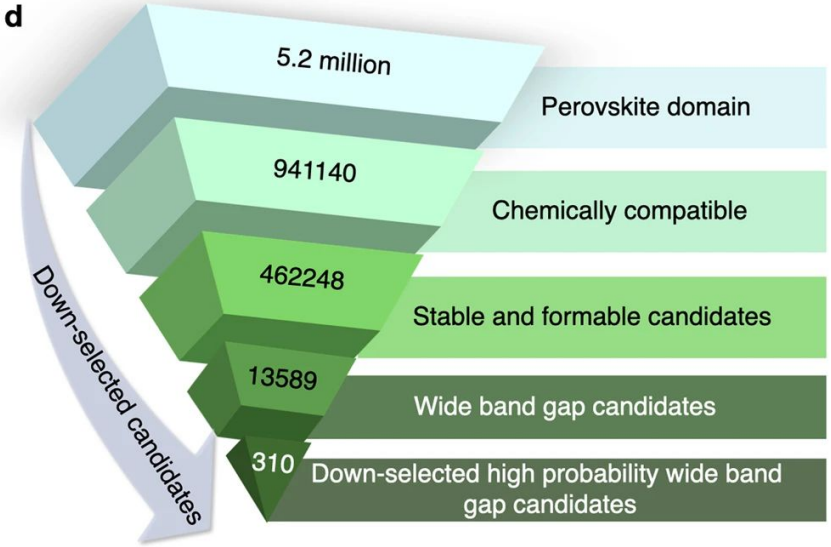
*DScribe: Library of descriptors for machine learning in materials science

Which features to consider?

- What accuracy do I need?
- At which scale your property is defined?
  - Site (i.e. defect formation energy)
  - Bond (i.e. migration barrier)
  - Structure (i.e. hardness)
- Size of the chemical space?
  - Several elements or the whole periodic table
- Diversity of crystal structures?
  - Fixed structural type? Not fixed?
- Do we know (expect) any relationship?
- How many samples do I have? 100? 100,000?

Example





Band gap predictions of double perovskite oxides using machine learning

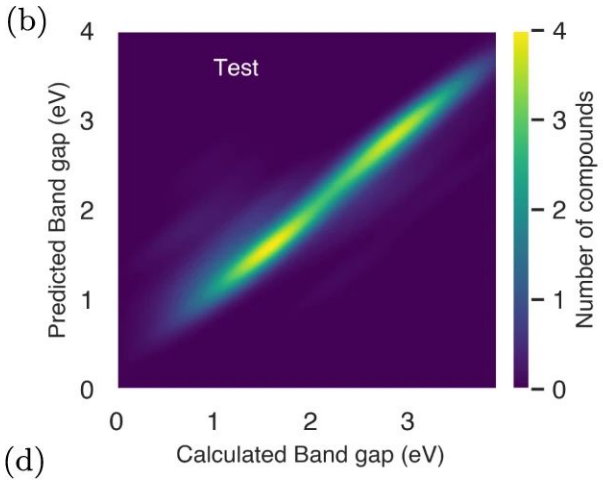
Considered features

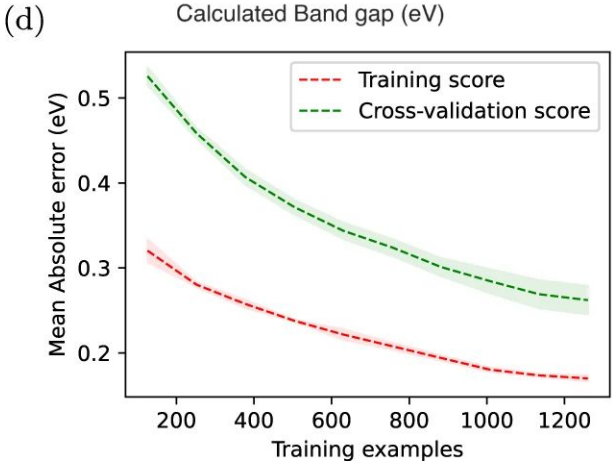
- Specific structural type - double perovskites
- Wide chemical space

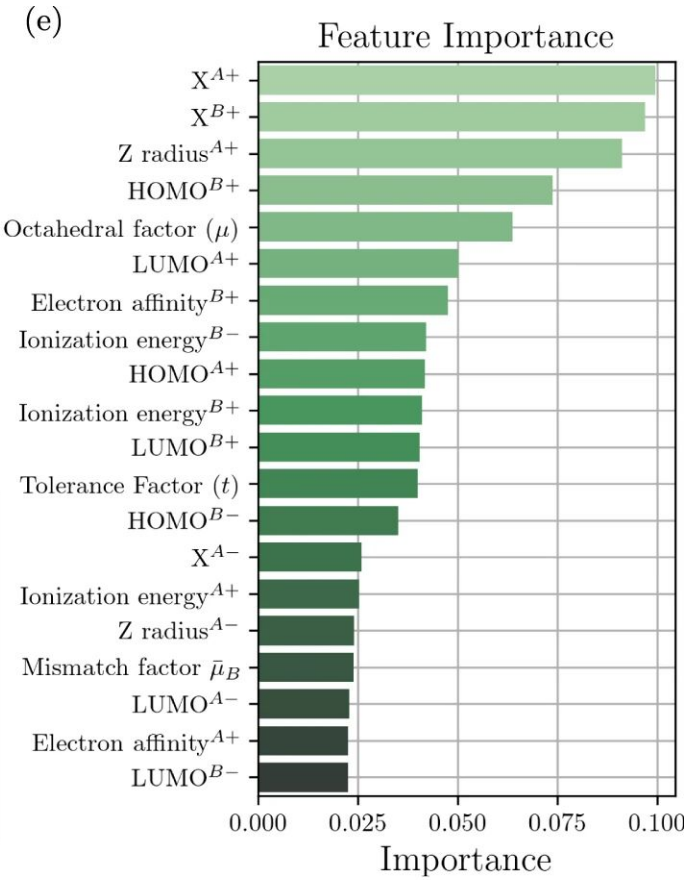
"…the intention was to use the simplest possible inputs and minimize computational overhead necessary to compile inputs while achieving high prediction accuracies by effectively incorporating notions of chemical similarity across different chemistries"

Abbreviation	Feature
Elemental	
HOMO	Highest Occupied Molecular Orbital (eV)
LUMO	Lowest Unoccupied Molecular Orbital (eV)
IE	Ionization energy (kJ/mol)
X	Pauling Electronegativity
Z radius	Zunger's Pseudopotential radius (a.u.)
EA	Electron affinity (kJ/mol)
Geometric	
t	Tolerance factor
μ	Octahedral factor
$\bar{\mu}$	Mismatch factor

Features importance for E_g prediction







Some typical mistakes in feature design for materials science

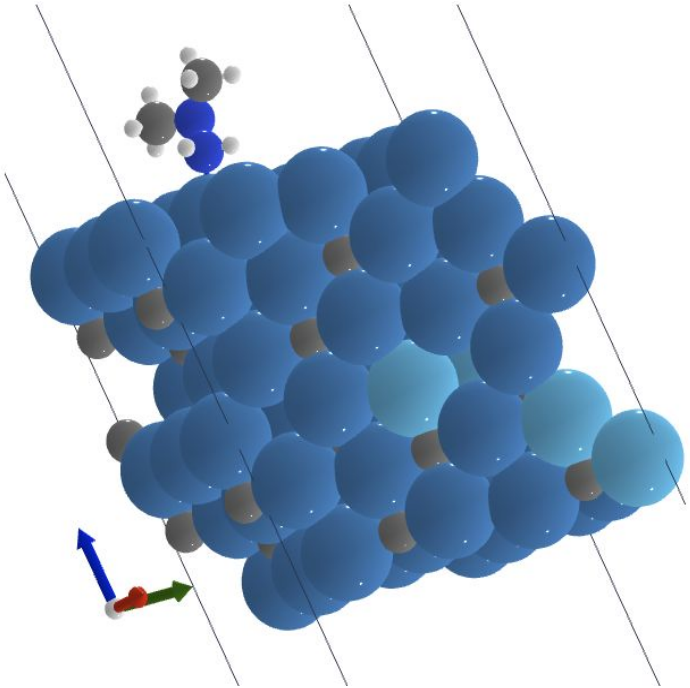
- Using high-dimensional features for small datasets – the curse of dimensionality
- Using features extracted from the calculations for the target property (see next slide) – target leakage or circular reasoning
- Using features from which target property can be algorithmically/deterministically defined – rediscovering arithmetic (e.g., use vbm and cbm for predicting band gap)
- Multicollinearity among features (e.g., ionic radii, covalent radii, etc) – difficult to separate the effect on the model
- No feature scaling (for some algorithms) or feature scaling on the entire dataset
- No domain expertise

Toy wrong study design example: Adsorption energy prediction

Task: Develop an ML model for predicting the adsorption energy of a set of molecules (CO, H_2, O_2, NH_3, etc) on catalyst surfaces (Co, Ni, Ti, Cu, etc)

Study design: DFT optimization of slab surfaces, DFT optimization of slab surfaces with adsorbates, computations of geometric features using DFT-optimized geometries for predicting adsorption energy

What's wrong: if you have DFT-optimized geometries you already have the target property, you don't need ML



Real wrong study design example: Crystal System Properties for Li-ion batteries

Task: Given a dataset of crystal structure descriptors. The crystal system (cubic, orthorhombic, tetragonal, etc) should be predicted.

Result: A mammoth work, EDA, huge pipeline, benchmarks, so on.
Claims: 90+% classification accuracy

What's wrong: A space group was used as a feature for predicting a crystal system.

Check this out:
https://www.kaggle.com/code/philippebillet/lithium-ion-batteries-stacking-and-importances#Final-Model-evaluation

Literature:

- Ward, Logan, et al. "Including crystal structure attributes in machine learning models of formation energies via Voronoi tessellations." Physical Review B 96.2 (2017): 024104. (https://doi.org/10.1103/PhysRevB.96.024104)
- Himanen, Lauri, et al. "DScribe: Library of descriptors for machine learning in materials science." Computer Physics Communications 247 (2020): 106949. (https://doi.org/10.1016/j.cpc.2019.106949)

Thank you for your attention!